AI MINI PROJECT REPORT · SEMESTER 2

Loan Approval Prediction

A supervised machine learning classification model for HDFC Bank Ltd.

Submitted by

Name	Roll No.
Agamjot Singh Dhingra	1025150317
Nikunj Singla	1025150306
Panshul Sharma	1025150302
Atharv Arora	1025150336
Aayush Goyal	1025150337

Submitted to: Dr. Jyoti

GitHub repository: https://github.com/agamjotdhingra/mini-project-1.git

Contents

1. Problem Statement & Objective

2. Dataset Source & Description

3. Data Understanding

4. Data Preprocessing

5. Exploratory Data Analysis (EDA) & Key Observations

6. Feature Selection / Engineering

7. Model Development

8. Evaluation & Model Comparison

9. Interpretation of Results

10. Limitations & Improvements

11. Conclusion

12. Project Flow Diagram

13. Code, Dataset & How to Run

1. Problem Statement & Objective

HDFC Bank Ltd. aims to improve its loan processing efficiency by predicting whether a loan application should be approved or rejected based on applicant details. Using a dataset containing information such as applicant income, loan amount, credit history, employment status, property area and other financial indicators, we develop a supervised machine learning classification model to categorise applications as Approved or Rejected.

Question	Our answer
Problem — what real-world problem are we solving?	Manual checking of every loan application is slow and inconsistent. The bank wants faster, data-driven screening.
Objective — what should the model achieve?	Predict whether a new application should be Approved or Rejected, and explain which factors matter most.
Inputs — what information will the model use?	Gender, marital status, dependents, education, self-employment, applicant & co-applicant income, loan amount, loan term, credit history, property area.
Output — what will the model predict?	Loan_Status: Approved (Y) or Rejected (N).

Type of ML problem

The output is a category (Approved / Rejected), not a number, and we have past applications with known outcomes. Therefore this is a supervised classification problem (binary classification). It is not regression (no numeric output) and not clustering (we do have a predefined target).

2. Dataset Source & Description

Source: the public “Loan Prediction” dataset (Kaggle / Analytics Vidhya Loan Prediction Problem), downloaded from a GitHub copy and stored in the repository as data/loan_data.csv. Each row is one past loan application.

Column	Type	Meaning
Loan_ID	Text	Unique application ID (dropped — no predictive value)
Gender	Categorical	Male / Female
Married	Categorical	Yes / No
Dependents	Categorical	0, 1, 2, 3+
Education	Categorical	Graduate / Not Graduate
Self_Employed	Categorical	Yes / No
ApplicantIncome	Numerical	Monthly income of applicant

CoapplicantIncome	Numerical	Monthly income of co-applicant
LoanAmount	Numerical	Loan amount (in thousands)
Loan_Amount_Term	Numerical	Term of loan in months
Credit_History	Numerical (0/1)	1 = meets credit guidelines, 0 = does not
Property_Area	Categorical	Urban / Semiurban / Rural
Loan_Status	Target	Y = Approved, N = Rejected

3. Data Understanding

Before any preprocessing we inspected what the data actually contains:

Check	Result
Shape	614 rows × 13 columns
Data types	8 categorical/text columns, 5 numerical columns
Missing values	Credit_History 50, Self_Employed 32, LoanAmount 22, Dependents 15, Loan_Amount_Term 14, Gender 13, Married 3
Duplicates	0 duplicate rows
Target	Loan_Status — 422 Approved (68.7%), 192 Rejected (31.3%) → imbalanced
Relevant features	All columns except Loan_ID are potentially useful

Why the imbalance matters: a useless model that approves everyone would already be about 69% accurate. Any model we build must clearly beat this baseline, and we must look beyond accuracy.

4. Data Preprocessing

The key question is not “Did we preprocess?” but “Why did we preprocess this way?”

Step	What we did	Why
Missing values (categorical)	Filled Gender, Married, Dependents, Self_Employed, Credit_History with the mode	Only a few rows are missing; the most common category is the most likely value. Dropping rows would lose ~20% of a small dataset.

Missing LoanAmount	Filled with the median	LoanAmount is right-skewed; the median is not pulled up by very large loans.
Missing Loan_Amount_Term	Filled with the mode (360 months)	512 of 600 known loans are 360 months.
Dependents “3+”	Converted to 3	Makes the column numeric.
Categorical encoding	Binary columns mapped to 1/0; Property_Area one-hot encoded	ML models only work with numbers; one-hot avoids implying Urban > Rural.
Outliers	Detected with box plots; kept, then log-transformed	High incomes are real applicants, not errors — removing them would lose real information.
Duplicates	Checked — none found	No action needed.
Scaling	StandardScaler, fitted on training data only, used for Logistic Regression	Logistic Regression is sensitive to feature scale; tree models are not. Fitting on train only prevents data leakage.
Loan_ID	Dropped	An identifier carries no information about approval.

5. Exploratory Data Analysis (EDA) & Key Observations

Every chart below answers a specific question. We avoided random plots.

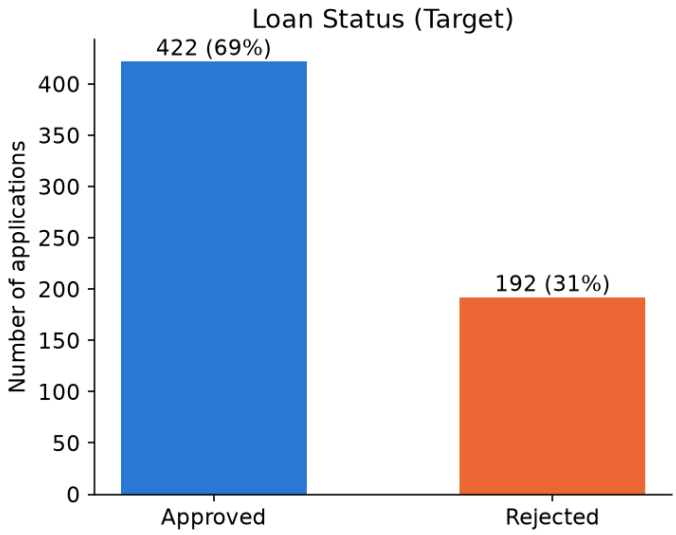


Figure 1 — Q: What does the target look like?

Observation: 422 approved vs 192 rejected. The target is imbalanced, so accuracy alone can be misleading.

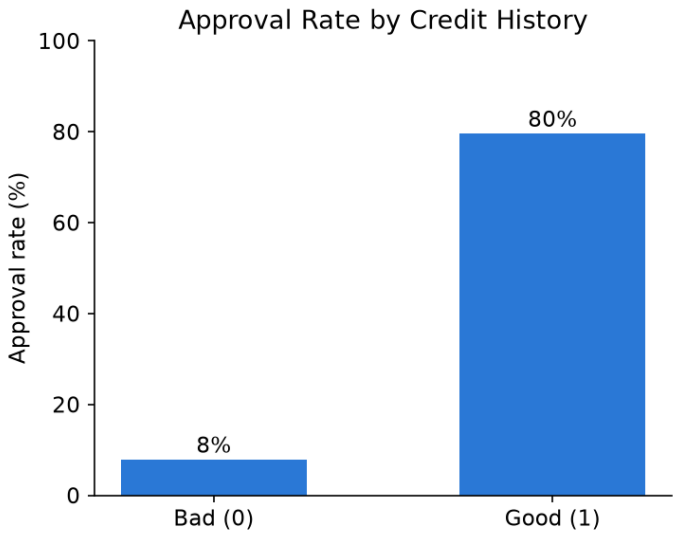


Figure 2 — Q: Does credit history affect approval?

Observation: Good credit history → 80% approved; bad credit history → only 8% approved. Credit history is the strongest single signal.

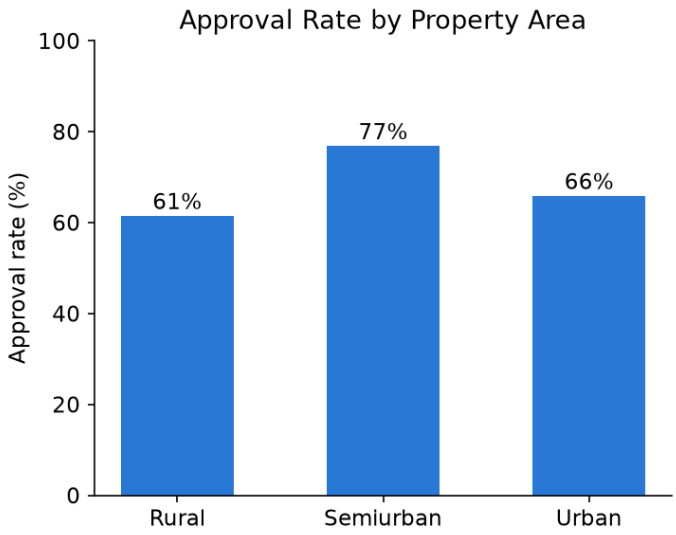


Figure 3 — Q: Does property area matter?

Observation: Semiurban applicants are approved most often (77%), Urban 66%, Rural least (61%).

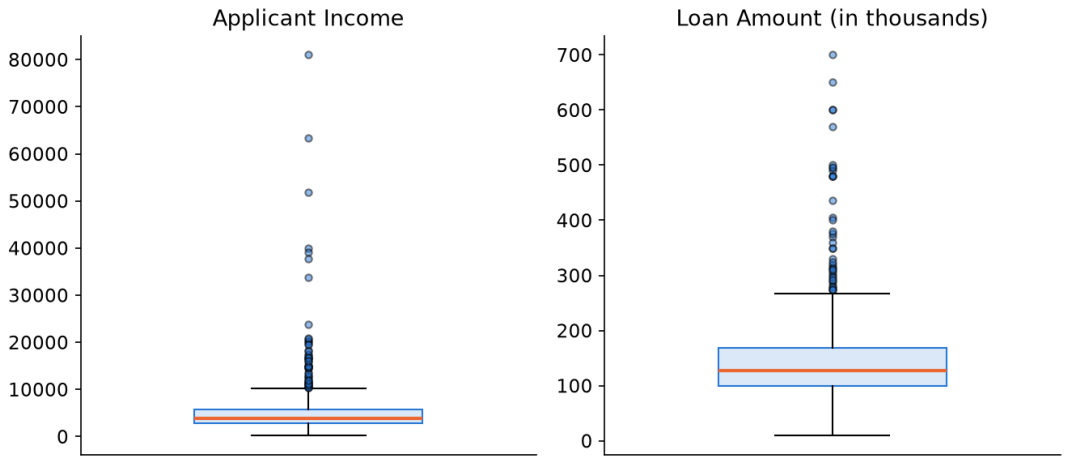


Figure 4 — Q: Are there outliers in income and loan amount?

Observation: Both are strongly right-skewed with many high outliers (applicant income goes up to 81,000). They are genuine, so we keep them and apply a log transform.

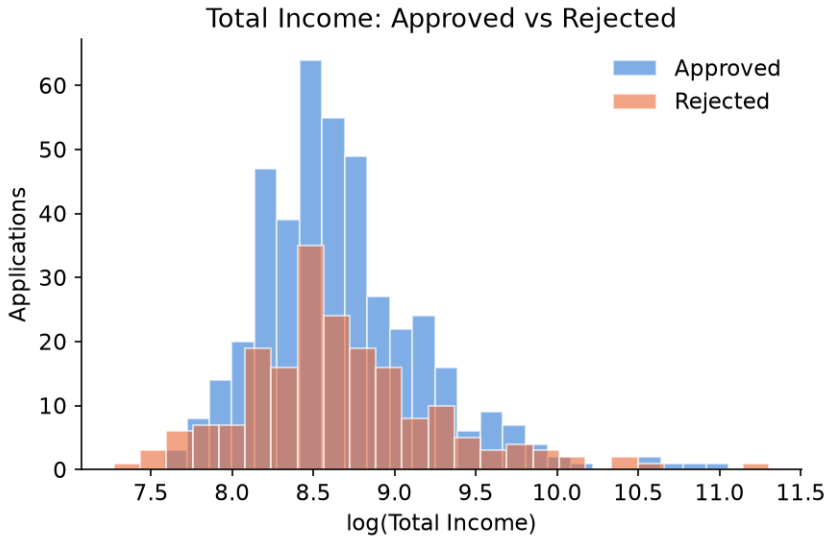


Figure 5 — Q: Do approved and rejected applicants differ in income?

Observation: The two distributions overlap almost completely — income alone does not decide approval. This is why we engineered ratio features.

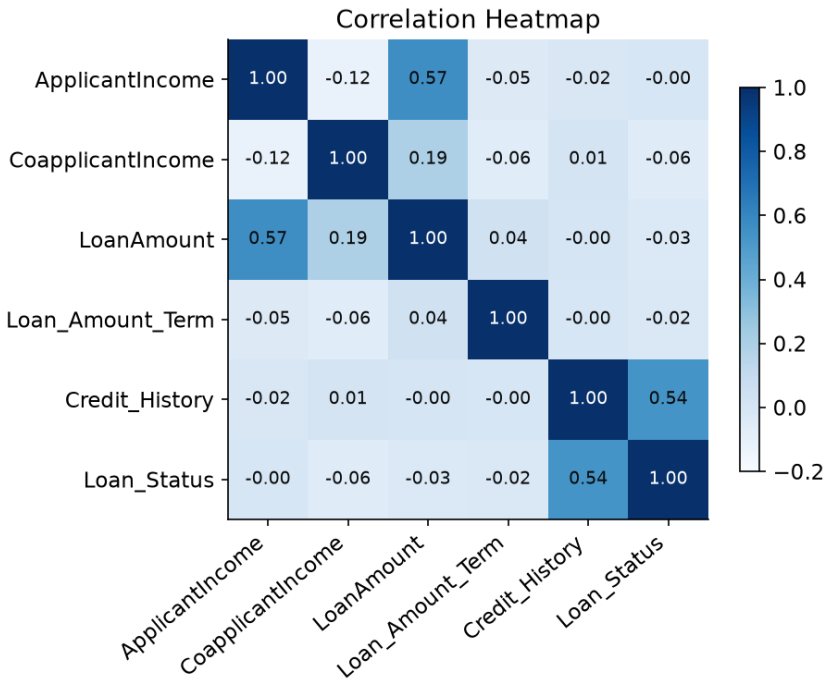


Figure 6 — Q: Which variables move together?

Observation: Credit_History has the highest correlation with Loan_Status (0.54). ApplicantIncome and LoanAmount are related (0.57): richer applicants ask for larger loans.

6. Feature Selection / Engineering

We asked: "Can the raw variables represent the problem well?" A bank cares about the applicant's ability to repay, which raw income alone does not capture. So we created meaningful new features:

New feature	Formula	Meaning
TotalIncome_log	log(ApplicantIncome + CoapplicantIncome)	Household income; log reduces skew from outliers
LoanAmount_log	log(LoanAmount)	Loan size with reduced skew
EMI	LoanAmount ÷ Loan_Amount_Term	Approximate monthly repayment burden
Loan_to_Income	LoanAmount ÷ TotalIncome	How large the loan is compared with income

Feature selection: after creating these, we removed the raw ApplicantIncome, CoapplicantIncome and LoanAmount columns because their information is already contained in the new features (avoids redundant variables). Final model inputs (13): Gender, Married, Dependents, Education, Self_Employed, Loan_Amount_Term, Credit_History, TotalIncome_log, LoanAmount_log, EMI, Loan_to_Income, Property_Area_Semiurban, Property_Area_Urban.

7. Model Development

The data was split 80% training / 20% testing with stratify=y (both parts keep the same 69/31 class ratio) and random_state=42 for reproducibility. We compared three sensible classification algorithms:

Model	Settings	Why we chose it
Logistic Regression	max_iter=1000, scaled inputs	Standard, simple baseline for yes/no problems; coefficients are easy to explain
Decision Tree	max_depth=4	Works like a loan officer's if-then rules; depth limited to avoid overfitting
Random Forest	200 trees, max_depth=5	Many trees voting together — usually more accurate and stable than one tree

We did not choose algorithms because they are popular; each fits a binary classification problem on small tabular data, and together they range from simple-and-explainable to more powerful.

8. Evaluation & Model Comparison

Metrics were chosen to match a classification problem with an imbalanced target:

- **Accuracy —** percentage of all predictions that are correct.
- **Precision —** of the applications we approved, how many were truly approved? (protects the bank from bad loans)
- **Recall —** of the truly approved applications, how many did we approve? (avoids losing good customers)
- **F1-score —** balance between precision and recall.
- **Confusion matrix —** shows exactly which mistakes the model makes.
- **5-fold cross-validation —** average accuracy over 5 different splits, so we don't trust one lucky split.

Model	Accuracy	Precision	Recall	F1-score	5-fold CV
Logistic Regression	85.4%	83.2%	98.8%	0.903	80.6%
Decision Tree	85.4%	85.3%	95.3%	0.900	79.3%
Random Forest	**87.0%**	**84.8%**	**98.8%**	**0.913**	**81.3%**
Baseline (approve all)	69.1%	—	—	—	—



Figure 7 — Model comparison on the test set

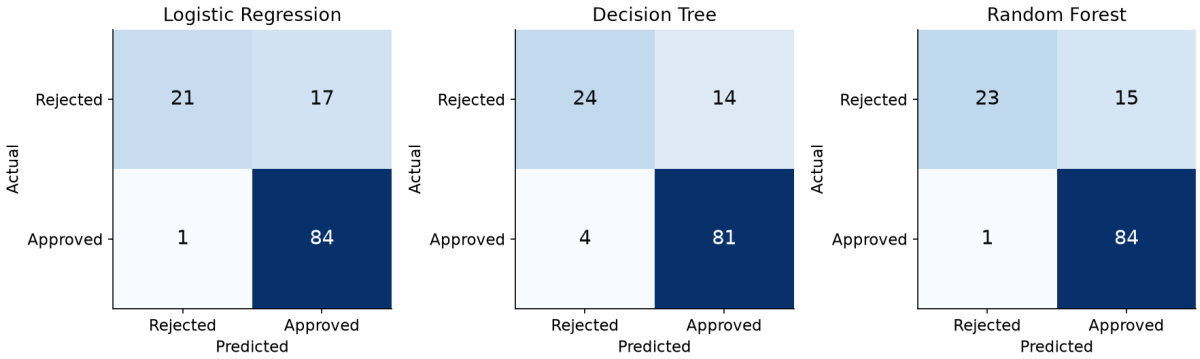


Figure 8 — Confusion matrices (test set, 123 applications)

Never one number alone: all models score 85–87% accuracy, but the confusion matrices show that Random Forest approves 84 of 85 genuinely approved applicants while catching only 23 of 38 rejections. So the model is very good at recognising good applicants but lets some risky ones through.

9. Interpretation of Results

What happened?

All three models beat the 69% baseline. Random Forest performed best on test accuracy (87.0%), F1-score (0.913) and cross-validation (81.3%). Logistic Regression was a close second.

Why?

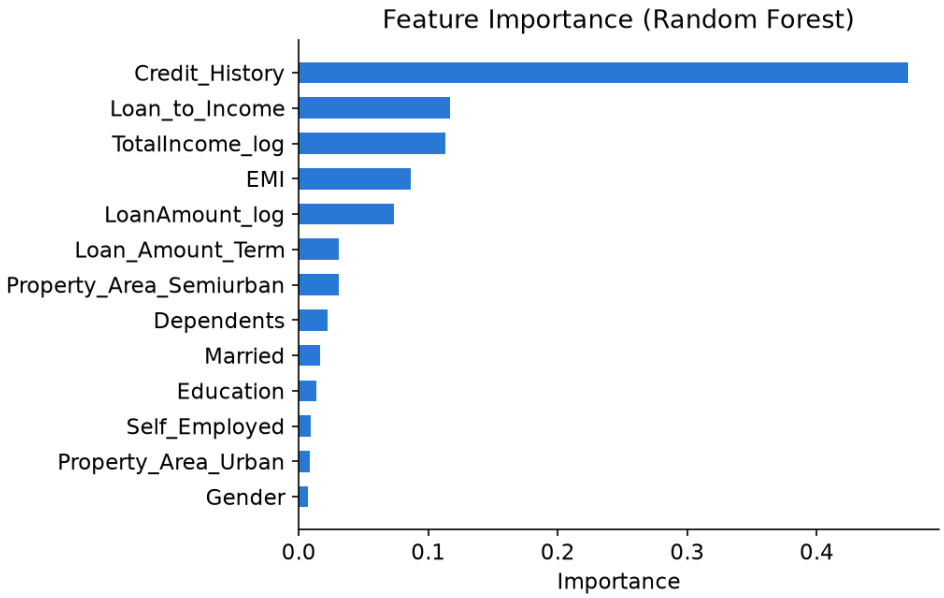


Figure 9 — Feature importance (Random Forest)

Credit_History alone accounts for 47% of the Random Forest's decisions, followed by Loan_to_Income (12%), TotalIncome_log (11%) and EMI (9%). Logistic Regression agrees: Credit_History has by far the largest positive coefficient (+1.24). Gender, self-employment and urban location contribute very little.

So what? (real-world meaning)

- Applicants with a good credit history and a reasonable loan-to-income ratio can be fast-tracked.
- Applicants with no or bad credit history should go to manual review.
- The model can act as a first-level screening tool for HDFC loan officers — prediction → insight → decision — not a replacement for human judgement.

10. Limitations & Improvements

Limitation	Possible improvement
Small dataset (614 rows) — results can change with a different split	Collect more historical applications
Imbalanced target — weaker at catching Rejected cases (15 risky applicants wrongly approved)	Use class_weight='balanced' or SMOTE; tune the decision threshold
Heavy reliance on Credit_History — may be unfair to first-time borrowers	Add alternative data (bank statements, repayment of utility bills)
Important banking features missing (CIBIL score, existing EMIs, age, employer)	Add these features
Default hyperparameters only lightly tuned	Use GridSearchCV; try gradient boosting (e.g., XGBoost)

11. Conclusion

We built a supervised classification model that predicts whether an HDFC Bank loan application should be approved. After cleaning the data, exploring it with question-driven visualisations and engineering repayment-related features, we compared Logistic Regression, Decision Tree and Random Forest. Random Forest was selected as the final model with 87% test accuracy and 81% cross-validated accuracy, well above the 69% baseline. Credit history is the deciding factor, followed by loan size relative to income. The model can speed up loan screening, but its weakness on rejected cases and its dependence on credit history mean it should support — not replace — loan officers.

12. Project Flow Diagram



Figure 10 — Complete project workflow (preprocessing and visualization highlighted)

13. Code, Dataset & How to Run

GitHub repository: https://github.com/agamjotdhingra/mini-project-1.git

File	Contents
Loan_Approval_Prediction.ipynb	Complete notebook with all outputs and charts
loan_approval.py	Same code as a plain Python script
data/loan_data.csv	Dataset (614 rows)
outputs/	All charts used in this report
requirements.txt	Python libraries needed

To run: install the libraries with pip install -r requirements.txt, then open the notebook in Jupyter or Google Colab and run all cells (or run python loan_approval.py).

Tools used: Python, pandas, NumPy, scikit-learn, matplotlib.